App Summary: Ontology-Grounded RAG Evaluation

What it is

A Python app for constrained-label diagnosis experiments that compares No-RAG versus ontology-grounded RAG on synthetic clinical chart text.
It builds or loads an ontology corpus, retrieves top-k ontology context, calls an LLM backend, and scores exact agreement against gold labels.

Who it is for

- Primary user/persona: ML or NLP researcher/developer running ontology-grounded evaluation pipelines on synthetic clinical datasets.
- Clinical end-user persona: Not found in repo.

What it does

- Loads ontology configuration (TCO, AI-RHEUM, diabetes, lung cancer) via src/onto_config.py.
- Builds or reuses corpus JSONL from BioPortal with offline fallback in src/tco_corpus.py.
- Creates FAISS or embedding retriever (with TF-IDF fallback) and caches artifacts in data/retriever_cache/.
- Builds bounded RAG prompt context from retrieved ontology terms in src/rag_context.py.
- Runs a unified LLM interface across Gemini, Hugging Face, OpenAI, or dry-run heuristic mode (src/llm_interface.py).
- Caches LLM prompt outputs and validates/coerces labels to allowed set plus NONE.
- Executes official exact-agreement evaluation and writes results/*.json,csv,md via src/eval_official.py.

How it works (repo-evidenced architecture)

- Inputs: data/label_set.json and data/synthetic_charts.csv.
- Ontology ingest: src/tco_corpus.py calls BioPortal API when available, else cached corpus file.
- Retrieval: src/retrievers.py indexes corpus docs and returns top-k hits per chart.
- Context assembly: src/rag_context.py formats retrieved docs into capped context text.
- Inference: src/llm_interface.py builds constrained prompt, selects backend by env vars, returns JSON prediction.
- Evaluation: src/eval_official.py computes No-RAG and RAG exact agreement, then writes results/ artifacts.
- Deployed service/API layer: Not found in repo.

How to run (minimal)

- Create env and install deps: python3 -m venv .venv && source .venv/bin/activate && python3 -m pip install -r requirements.txt
- Optional backend keys: set GOOGLE_API_KEY or OPENAI_API_KEY, or set USE_HUGGINGFACE=true for local model.
- Run official evaluation: python3 evaluate.py
- Optional ontology switch: python3 -m src.eval_official --ontology-key ai_rheum